



NUS
National University
of Singapore

NUS
BUSINESS
SCHOOL

DBA3803

Predictive Analytics in Business

Dr. Hong Ming Tan



Netflix

Predictive Analytics in Streaming Services

SA1 - Group 7

Name	Matriculation Number
Pierce Toh Yixian	A0262262L
Pham Thuy Hien	A0338245B
Hsu Serena Katherine	A0287741U
Oliwer Bednarz	A0338058X
Lee Taehyung	A0302747E

Table of Contents

1. Abstract	1
2. Introduction	1
2.1 Company Background	1
2.2 Problem Statement	1
3. Data	1
3.1 Data Profile	1
3.2 Data Pre-Processing	2
4. Methodology	3
4.1 Train-Test Split	3
4.2 Candidate Models	3
4.3 Cross-Validation and Hyperparameter Tuning	4
4.4 Test Set Evaluation	5
5. Results	6
5.1 Model Evaluation via ROC-AUC, Precision & Recall	6
5.2 Feature Importance	8
5.3 Decision Tree Visualisation	9
5.4 Churn Rate by Features	9
6. Discussion	10
6.1 Key Findings	10
6.2 Limitations	11
6.2.1 Data Limitations	11
6.2.2 Model Limitations	11
6.3 Recommendations	11
7. Conclusion	11
8. References	12
9. Appendix	12

1. Abstract

This report created prediction models to predict the probability of customer churn rate within a Netflix customer base. By implementing a dataset of 5,000 customers acquired via Kaggle, we analyzed using a variety of machine learning models, including Logistic Regression, Lasso Logistic Regression, Decision Tree, and Random Forest, to determine the best-fit prediction model. To evaluate the model's performance, we evaluated model performance using ROC-AUC, Precision & Recall, and further interpreted results using Decision Tree visualisation to verify the chosen model's classification accuracy. As a result, Gradient Boosting performs the best, with an ROC of 0.999 and an average precision score of 0.999. This model suggests that the most important predictors are daily watch time, total watch hours, number of profiles, and last login days. These results demonstrate how important user engagement behavior is in assessing subscription cancellations. We suggested that Netflix should prioritise in-app activities engagement and identify high-risk customer tiers to assist focused retention tactics.

2. Introduction

2.1 Company Background

Netflix is a subscription-based streaming service that provides access to documentaries, television series, and movies. The company operates through a Business-to-Customer (B2C) model, generating revenue from monthly subscription fees. Netflix differentiates itself by having a strong global brand, a diverse range of international movie library, personalised recommendations, and multiple access across devices. Customer retention is a critical factor to Netflix's long-term performance as it contributes to revenue stability, sustainability growth, and investment in movie streaming and platform development. We selected Netflix because the company strongly represents the correlation between customer retention rate and business performance. By having access to a large customer base of Netflix, we can implement predictive models to identify patterns in subscribers' behavior, estimate the likelihood of churn, and give recommendations on best-fit models that help the firm determine high-risk customer tiers for their operation management and business decisions.

2.2 Problem Statement

In the case of subscription models, customer churn becomes a major issue for firms because it reduces their periodic income and increases the costs incurred in attracting new customers. Since Netflix's revenue largely relies on customers' subscriptions, its recent price increase of SGD\$4 during early 2026 (Alavanza, 2025) and its updated policy restricting account sharing (NIQ, 2024) may elevate customer churn risk, further reducing subscribers. As customers become more price-sensitive, it can create pressure on Netflix to retain its loyal customer base, particularly significant in a highly competitive streaming market. Given the sheer size of its client base, even marginal changes in turnover will significantly affect Netflix's bottom line.

In terms of data usage, we used the Netflix Customer Churn Dataset on Kaggle, created by Abdul Wadood, updated 9 months ago. This dataset contains 5,000 records with 14 engineered features that are particularly designed for churn prediction modelling and classification, customer behavior modelling, and segmentation. Although the dataset is synthetic and may not reflect the Netflix customer behavior realistically, it remains useful for our report because we can use the dataset to demonstrate the analysis of churn drivers, develop analytical insights through predictive models including Logistic Regression, Ridge Logistic Regression, Lasso Logistic Regression, Elastic Net Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting in the context that closely related to Netflix's business model. The key variables present in our dataset include the average watch time per day, number of watch hours, number of profiles, number of last login days, subscription types (Basic, Standard, Premium), and payment methods.

3. Data

3.1 Data Profile

The dataset used for this study is sourced from Kaggle and contains the following types of data, as shown in Figure 1. It simulates customer behaviour for Netflix, with 5,000 total records detailing characteristics of the customer profile and their usage statistics on the streaming platform.

Column	Description / Profile
customer_id	Unique identifier (Primary key)
age	Discrete variable with min 18, max 70, and average of 43
gender	Categorical variable (3): Male / Female / Other
subscription_type	Categorical variable (3): Basic / Standard / Premium
watch_hours	Continuous variable with min 0.01, max 110.4, and average of 11.6
last_login_days	Discrete variable with min 0, max 60, and average of 30
region	Categorical variable (6): Africa / Europe / Asia / Oceania / South America / North America
device	Categorical variable (5): TV / Mobile / Laptop / Desktop / Tablet
monthly_fee	Corresponds to subscription type. Discrete variable: 8.99 / 13.99 / 17.99
payment_method	Categorical variable (5) : Gift Card / Crypto / Debit Card / PayPal / Credit Card
number_of_profiles	Discrete variable 1-5
avg_watch_time_per_day	Lifetime Avg. Continuous variable with min 0.0, max 98.42, and average of 0.87
favorite_genre	Categorical variable (7): Action, Sci-Fi, Drama, Horror, Romance, Comedy, Documentary
churned	Binary variable: 0 / 1

Figure 1: Types of Data in the Dataset

3.2 Data Pre-Processing

Prior to processing the data, the dataset was checked for missing values, duplicates, and class balance. No missing values or duplicate records were identified across all 5,000 records.

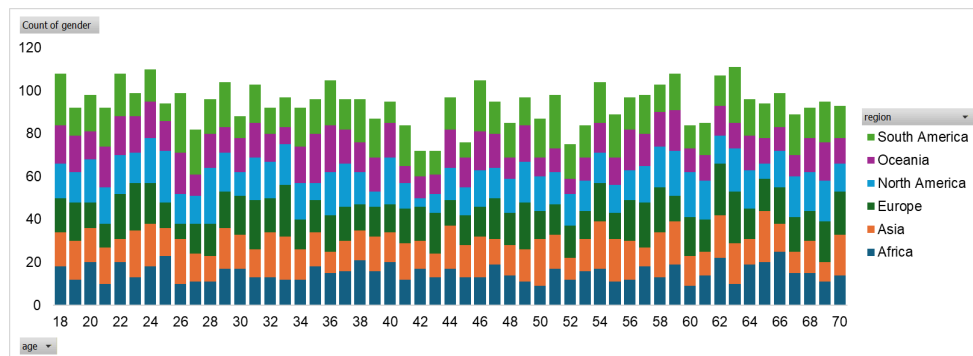


Figure 2: Distribution of Age Counts across Regions

Other	1,635	Basic	1,661	Africa	803
Female	1,711	Standard	1,646	Europe	867
Male	1,654	Premium	1,693	Asia	841
Total	5,000	Total	5,000	Oceania	765
				South Ame	873
				North Ame	851
				Total	5,000

Figure 3: Distribution of Select Features: Gender, Subscription Type, and Region

The target classes were also checked to ensure that they were broadly balanced across several characteristics. As shown in Figures 2-3, the dataset contained a balanced spread of values across various customer account demographics, hence it was concluded that no resampling was required.

For the purpose of training the predictive model, the column customer_id was subsequently removed because it is only an identifier and carries no predictive value. Column monthly_fee was also excluded as the fees customers paid were a discrete variable directly based on the subscription_type and were therefore redundant in the model (4. Prepare modeling dataset, Appendix A1)

customer_id	age	gender	subscription_type	watch_hours	last_login_days	region	device	monthly_fee	churned	payment_method	number_of_profiles	avg_watch_time_per_day	favorite_genre
a9b75100-82	51	Other	Basic	14.73	29	Africa	TV	8.99	1	Gift Card	1	0.49	Action
49a5dfd9-7e	47	Other	Standard	0.7	19	Europe	Mobile	13.99	1	Gift Card	5	0.03	Sci-Fi
4d71f6ce-fca	27	Female	Standard	16.32	10	Asia	TV	13.99	0	Crypto	2	1.48	Drama
d3c72c38-63	53	Other	Premium	4.51	12	Oceania	TV	17.99	1	Crypto	2	0.35	Horror
4e265c34-10	56	Other	Standard	1.89	13	Africa	Mobile	13.99	1	Crypto	2	0.13	Action
d8079475-5b	58	Female	Standard	13.8	26	Oceania	Mobile	13.99	0	Debit Card	3	0.51	Action
8e63450a-13	48	Other	Basic	13.83	20	Asia	TV	8.99	0	Gift Card	5	0.66	Romance
02387681-8c	51	Male	Basic	14.3	56	Europe	Mobile	8.99	1	Gift Card	1	0.25	Action
0bcaad0c-54	45	Other	Basic	9.98	10	Asia	Mobile	8.99	0	PayPal	3	0.91	Romance
eeae6439e-8c	32	Other	Premium	2.22	34	Europe	TV	17.99	1	Debit Card	1	0.06	Drama
45a03ce3-26	26	Male	Premium	15.42	36	Asia	Laptop	17.99	0	Debit Card	4	0.42	Comedy

Figure 4: First 10 Rows of Data in the Dataset

Additionally, the data were further processed for different predictive model uses. For Logistic Regression and Lasso Logistic Regression, numerical variables (age, watch_hours, last_login_days, number_of_profiles, and avg_watch_time_per_day) were standardised, and categorical variables (gender, subscription_type, region, device, payment_method, and favorite_genre) were one-hot encoded, as these models are sensitive to differences in scale. For decision tree, random forest, and gradient boosting, numerical variables were kept in their original scale, while categorical variables were similarly one-hot encoded, since tree-based models do not require scaling. These preprocessing steps were implemented through pipeline objects (5. Preprocessing, Appendix A1) to ensure the consistent application of transformations without data leakage.

While the dataset has no missing values or duplicates, its synthetic nature is an important limitation to take into consideration. Real-world Netflix customer data would likely be noisier, less balanced, and shaped by more complex behavioural patterns. As a result, the model's performance may overstate how well it would generalise in practice.

4. Methodology

As stated, this is a binary classification problem to understand and predict whether a customer churned (1) or remained subscribed (0). Therefore, 7 supervised learning methods were applied to compare different predictive methods: Logistic Regression (Baseline), Lasso Logistic Regression, Ridge Logistic Regression, Elastic Net Logistic Regression, Decision Tree, Random Forest, and Gradient Boosting. Logistic Regression is used as an interpretable baseline, while Lasso tests whether a smaller set of predictors can explain churn effectively. Decision Tree aims to capture threshold-based decision rules, and Random Forest and Gradient Boosting assess whether ensemble methods can better identify nonlinear relationships.

4.1 Train-Test Split

The data was split into 70% training and 30% test sets using stratified sampling to preserve the class distribution across both data sets, ensuring that both are representative of the entire dataset.

```
# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("\nTrain-test split completed.")
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)
```

Train-test split completed.
X_train shape: (3500, 11)
X_test shape: (1500, 11)
y_train shape: (3500,)
y_test shape: (1500,)

Figure 5: Train-Test Split Code

4.2 Candidate Models

Logistic Regression was used as the baseline model, while the remaining models were tuned using GridSearchCV with 5-fold cross-validation. There are 3 regularised variants. Firstly, Ridge Logistic Regression applies an L2 penalty, penalising large coefficients without eliminating them, keeping all predictors in the event of correlation between them. Secondly, Lasso Logistic Regression applies an L1 penalty, driving certain coefficients to zero, allowing us to perform feature selection. Thirdly, Elastic Net Logistic Regression combines the L1 and L2 penalties to provide a middle ground between Ridge and Lasso.

We also employ a Decision Tree model, which partitions the data into interpretable regions based on threshold values. A Random Forest model is also included, which creates an ensemble of Decision Trees using bootstrapped samples and each tree using a random subset of features at each split before aggregating

predictions to reduce overfitting. Finally, a Gradient Boosting model is used, which fits Decision Trees sequentially, iteratively building on the error of the previous tree to yield high-accuracy predictions.

```
log_reg_pipeline = Pipeline([
    ("preprocessor", linear_preprocessor),
    ("model", LogisticRegression(max_iter=1000, random_state=42))
])

ridge_pipeline = Pipeline([
    ("preprocessor", linear_preprocessor),
    ("model", LogisticRegression(
        penalty="l2",
        solver="lbfgs",
        max_iter=1000,
        random_state=42
    ))
])

lasso_pipeline = Pipeline([
    ("preprocessor", linear_preprocessor),
    ("model", LogisticRegression(
        penalty="l1",
        solver="liblinear",
        max_iter=1000,
        random_state=42
    ))
])

elastic_pipeline = Pipeline([
    ("preprocessor", linear_preprocessor),
    ("model", LogisticRegression(
        penalty="elasticnet",
        solver="saga",
        l1_ratio=0.5,
        max_iter=3000,
        random_state=42
    ))
])

fixed_dt_pipeline = Pipeline([
    ("preprocessor", tree_preprocessor),
    ("model", DecisionTreeClassifier(
        max_depth=5,
        min_samples_split=20,
        min_samples_leaf=10,
        random_state=42
    ))
])

dt_pipeline = Pipeline([
    ("preprocessor", tree_preprocessor),
    ("model", DecisionTreeClassifier(random_state=42))
])

rf_pipeline = Pipeline([
    ("preprocessor", tree_preprocessor),
    ("model", RandomForestClassifier(random_state=42))
])

gb_pipeline = Pipeline([
    ("preprocessor", tree_preprocessor),
    ("model", GradientBoostingClassifier(random_state=42))
])
```

Figure 6: Candidate Model Pipelines Code

4.3 Cross-Validation and Hyperparameter Tuning

To assess individual model performance, a 5-fold stratified cross-validation is used. The training data is divided into 5 equal, class-balanced folds, where the model is trained on the first 4 and evaluated on the remaining fold before the performance of the model is averaged across 5 iterations. Cross-validation ensures that the performance is more reliably measured, rather than using only a single train-validation split. 5 metrics are tracked during this cross-validation: accuracy, precision, recall, F1 score, and ROC-AUC.

```
kfold = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1",
    "roc_auc": "roc_auc"
}

def evaluate_cv_model(pipeline, name, X_data, y_data):
    scores = cross_validate(
        pipeline,
        X_data,
        y_data,
        cv=kfold,
        scoring=scoring,
        return_train_score=False
    )

    print(f"\n=== {name} Cross Validation ===")
    for metric in scoring.keys():
        print(f"{metric}: {np.mean(scores['test_' + metric]):.4f}")

evaluate_cv_model(ridge_pipeline, "Logistic Regression (Ridge)", X, y)
evaluate_cv_model(lasso_pipeline, "Logistic Regression (Lasso)", X, y)
evaluate_cv_model(elastic_pipeline, "Logistic Regression (Elastic Net)", X, y)
evaluate_cv_model(fixed_dt_pipeline, "Decision Tree (Fixed Parameters)", X, y)
```

Figure 7: 5-Fold Stratified Cross-Validation Code

We then employ GridSearchCV to automate hyperparameter tuning for 5 out of 7 models via cross-validation, to determine optimal hyperparameters for each model. Hyperparameters were selected based on optimising ROC-AUC, which was also used as the primary metric for comparing model performance. For Lasso, the inverse regularisation strength (C) ranged from 0.01 to 10.0, and for Elastic Net, the same C range is used together with the L1 ratio. The Decision Tree tuning looks at maximum tree depth, minimum samples required to split a node, and minimum samples required at a leaf node, to sensitise the model's resistance to overfitting and complexity in depth. For the Random Forest, similar hyperparameters are included together with the number of trees. For Gradient Boosting, the learning rate is also evaluated, which controls the successive contribution of each tree.

```

lasso_param_grid = {
    "model__C": [0.01, 0.1, 1.0, 5.0, 10.0]
}

elastic_param_grid = {
    "model__C": [0.01, 0.1, 1.0, 5.0],
    "model__l1_ratio": [0.2, 0.5, 0.8]
}

dt_param_grid = {
    "model__max_depth": [3, 5, 7, 10, None],
    "model__min_samples_split": [2, 5, 10, 20],
    "model__min_samples_leaf": [1, 2, 5]
}

rf_param_grid = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [5, 10, None],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 2, 5]
}

gb_param_grid = {
    "model__n_estimators": [100, 200],
    "model__learning_rate": [0.01, 0.05, 0.1],
    "model__max_depth": [2, 3, 4]
}

# Baseline Logistic regression CV AUC
log_reg_pipeline.fit(X_train, y_train)
log_cv_auc = cross_val_score(
    log_reg_pipeline,
    X_train,
    y_train,
    cv=5,
    scoring="roc_auc"
).mean()

# Ridge CV AUC
ridge_pipeline.fit(X_train, y_train)
ridge_cv_auc = cross_val_score(
    ridge_pipeline,
    X_train,
    y_train,
    cv=5,
    scoring="roc_auc"
).mean()

# Lasso tuning
lasso_grid = GridSearchCV(
    lasso_pipeline,
    lasso_param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
lasso_grid.fit(X_train, y_train)
best_lasso = lasso_grid.best_estimator_
print("Best Lasso params:", lasso_grid.best_params_)

# Elastic Net tuning
elastic_grid = GridSearchCV(
    elastic_pipeline,
    elastic_param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
elastic_grid.fit(X_train, y_train)
best_elastic = elastic_grid.best_estimator_
print("Best Elastic Net params:", elastic_grid.best_params_)

# Decision Tree tuning
dt_grid = GridSearchCV(
    dt_pipeline,
    dt_param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
dt_grid.fit(X_train, y_train)
best_dt = dt_grid.best_estimator_
print("Best Decision Tree params:", dt_grid.best_params_)

# Random Forest tuning
rf_grid = GridSearchCV(
    rf_pipeline,
    rf_param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
rf_grid.fit(X_train, y_train)
best_rf = rf_grid.best_estimator_
print("Best Random Forest params:", rf_grid.best_params_)

# Gradient Boosting tuning
gb_grid = GridSearchCV(
    gb_pipeline,
    gb_param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
gb_grid.fit(X_train, y_train)
best_gb = gb_grid.best_estimator_
print("Best Gradient Boosting params:", gb_grid.best_params_)

```

Figure 8: Hyperparameter Tuning Code

Using GridSearchCV, the optimal hyperparameters obtained are as follows:

```

Best Lasso params: {'model__C': 10.0}

Best Elastic Net params: {'model__C': 5.0, 'model__l1_ratio': 0.8}
Best Decision Tree params: {'model__max_depth': 10, 'model__min_samples_leaf': 5, 'model__min_samples_split': 20}
Best Random Forest params: {'model__max_depth': None, 'model__min_samples_leaf': 1, 'model__min_samples_split': 2, 'model__n_estimators': 200}
Best Gradient Boosting params: {'model__learning_rate': 0.1, 'model__max_depth': 4, 'model__n_estimators': 200}

```

Figure 9: Optimal Hyperparameters

The baseline and Ridge Logistic Regression models are evaluated without grid search cross-validation, as their default regularisation is simple and not essential to tune.

4.4 Test Set Evaluation

All 7 tuned/fitted models are evaluated on the test set with additional evaluation metrics including accuracy, precision, recall, F1-score, ROC-AUC, and supported by confusion matrices and ROC curves. Models are ranked by test ROC-AUC, and the best model is identified together with its confusion matrix (Figure 14). To better illustrate the comparison between the discrimination performance of the models, their ROC curves are plotted together on the same chart (Figure 12).

```
def evaluate_model(model, X_test, y_test, model_name, cv_auc=np.nan):
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    return {
        "model": model_name,
        "cv_auc": cv_auc,
        "test_accuracy": accuracy_score(y_test, y_pred),
        "test_precision": precision_score(y_test, y_pred),
        "test_recall": recall_score(y_test, y_pred),
        "test_f1": f1_score(y_test, y_pred),
        "test_roc_auc": roc_auc_score(y_test, y_prob),
        "confusion_matrix": confusion_matrix(y_test, y_pred)
    }

results_list = []

results_list.append(
    evaluate_model(log_reg_pipeline, X_test, y_test, "Logistic Regression", log_cv_auc)
)
results_list.append(
    evaluate_model(ridge_pipeline, X_test, y_test, "Ridge Logistic Regression", ridge_cv_auc)
)
results_list.append(
    evaluate_model(best_lasso, X_test, y_test, "Lasso Logistic Regression", lasso_grid.best_score_)
)
results_list.append(
    evaluate_model(best_elastic, X_test, y_test, "Elastic Net Logistic Regression", elastic_grid.best_score_)
)
results_list.append(
    evaluate_model(best_dt, X_test, y_test, "Decision Tree", dt_grid.best_score_)
)
results_list.append(
    evaluate_model(best_rf, X_test, y_test, "Random Forest", rf_grid.best_score_)
)
results_list.append(
    evaluate_model(best_gb, X_test, y_test, "Gradient Boosting", gb_grid.best_score_)
)

results_df = pd.DataFrame(results_list).sort_values(by="test_roc_auc", ascending=False)

print("\nModel comparison table:")
print(results_df)
```

Figure 10: Test Set Evaluation Code

To compare their ability in avoiding overfitting, their train-vs-test ROC-AUC is also compared (Figure 11). A large gap between training and test ROC-AUC would suggest a higher degree of overfitting, and a limited training effect where the model had memorised the data instead of learning patterns.

```
def train_test_auc(model, X_train, y_train, X_test, y_test, model_name):
    train_prob = model.predict_proba(X_train)[:, 1]
    test_prob = model.predict_proba(X_test)[:, 1]

    train_auc = roc_auc_score(y_train, train_prob)
    test_auc = roc_auc_score(y_test, test_prob)

    print(f"\n{model_name}")
    print(f"Train ROC-AUC: {train_auc:.4f}")
    print(f"Test ROC-AUC: {test_auc:.4f}")

for name, model in model_dict.items():
    train_test_auc(model, X_train, y_train, X_test, y_test, name)
```

```
Logistic Regression
Train ROC-AUC: 0.9686
Test ROC-AUC: 0.9644

Ridge Logistic Regression
Train ROC-AUC: 0.9686
Test ROC-AUC: 0.9644

Lasso Logistic Regression
Train ROC-AUC: 0.9701
Test ROC-AUC: 0.9671

Elastic Net Logistic Regression
Train ROC-AUC: 0.9697
Test ROC-AUC: 0.9664

Decision Tree
Train ROC-AUC: 0.9992
Test ROC-AUC: 0.9907

Random Forest
Train ROC-AUC: 1.0000
Test ROC-AUC: 0.9948

Gradient Boosting
Train ROC-AUC: 1.0000
Test ROC-AUC: 0.9989
```

Figure 11: Train-vs-Test ROC-AUC Comparison Across Models

To fully interpret the results and understand the drivers of churn, coefficient estimates were examined for the Regression models, decision rules were reviewed for the Decision Tree, and feature importance scores were analysed for Random Forest and Gradient Boosting. The results are covered in depth in the next section.

5. Results

5.1 Model Evaluation via ROC-AUC, Precision & Recall

ROC-AUC (Area Under the Receiver Operating Characteristic Curve) was used to evaluate how well each model can evaluate and distinguish between churned and retained customers across classification models.

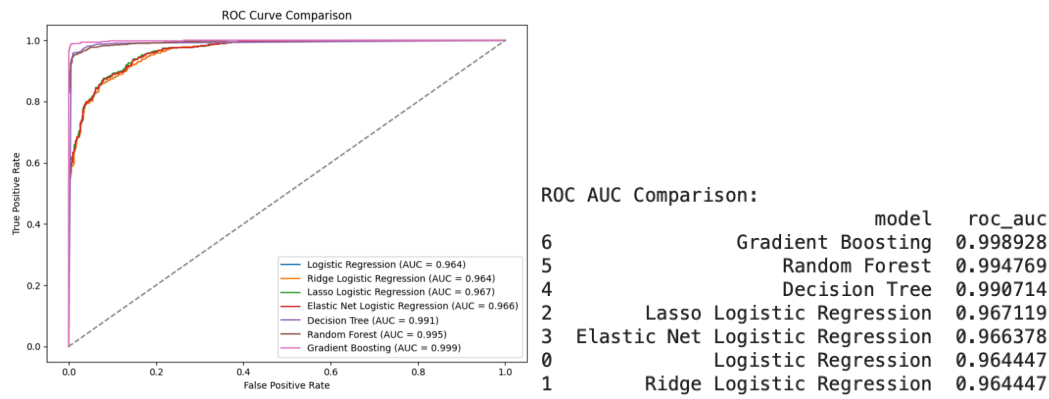


Figure 12: ROC-AUC Curve and Comparison Table

According to the ROC-AUC comparative, all models demonstrated strong classification performance, with Gradient Boosting achieving the highest test at 0.9989, followed by Random Forest and Decision Tree. This suggests that the relationship between the predictors and churn is not purely linear, and the more flexible the model, the better it can capture the underlying churn patterns in the data set.

We then used Precision and Recall to identify the positive class, as an imbalanced dataset's accuracy can be misleading. Precision shows how many predicted positive cases are actually correct (avoiding false positives), while recall shows how many actual positive cases (avoiding false negatives) are identified as real churners.

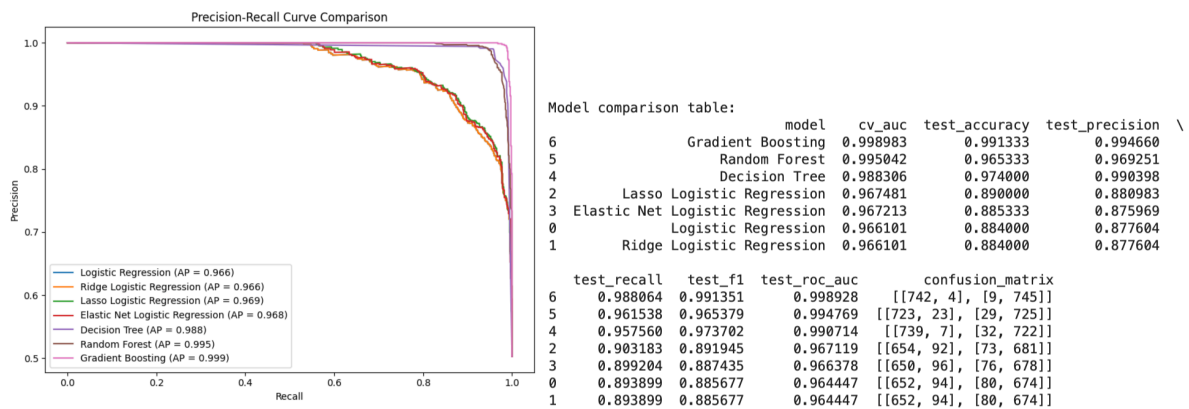


Figure 13: Precision Recall Curve and Comparison Table

Similarly, in the precision-recall comparison model, Gradient Boosting performs the best with an average precision score of 0.999, while Random Forest and Decision Tree also perform strongly at 0.995 and 0.988, respectively. We conducted a model comparison table to analyse the strongest model overall and observed that Gradient Boosting achieves a test accuracy of 0.9913, test precision of 0.9947, test recall of 0.9881, and test F1-score of 0.9914, which demonstrates the best performance across all other evaluation metrics.

These results provide strong evidence for selecting Gradient Boosting as the final model for our interpretation. Since precision-recall performance can help Netflix identify high-risk customers accurately without targeting too many customers who would have stayed anyway, we believe that Gradient Boosting appears to be the most suitable model within this dataset to make churn prediction.

A confusion matrix is used to show the number of churners that were correctly identified and non-churners classified, as well as prediction errors. In terms of classification measures, precision shows the reliability of predictions, recall shows how well the model captures actual positive cases, and F1 combines precision and recall into one single metric, allowing us to judge how well the model handles churn prediction.

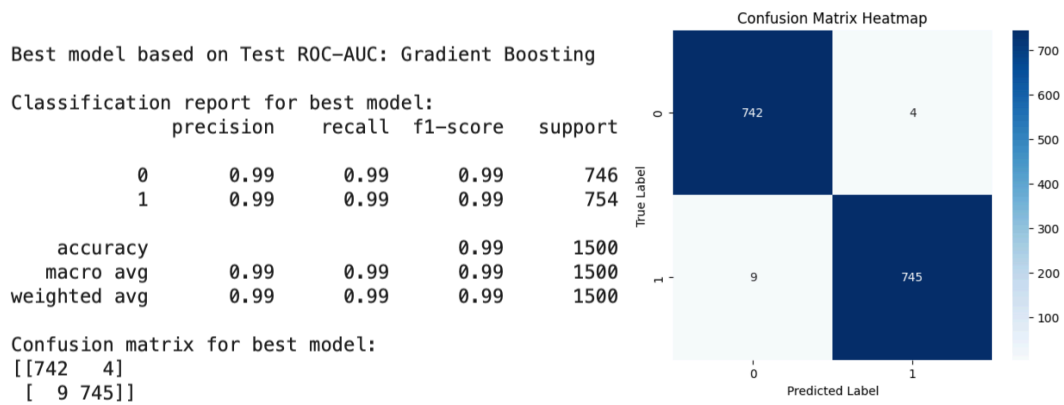


Figure 14: Classification report for Gradient Boosting

Accordingly, the confusion matrix for the selected Gradient Boosting shows that it classified the test data with very few errors, which correctly identified 742 retained customers and 745 churned customers, while misclassifying only 4 retained customers as churned and 9 churned customers as retained. Precision, recall, and F1 are all 0.99, with an accuracy of 0.99 on 1,500 test observations. As a result, it indicates the model performs highly effectively on this dataset at separating churned from retained customers.

From Netflix's perspective, using this classification model suggests strong predictive performance, given the low number of classification errors, potentially allowing the firm to save time and focus on high-risk customers more efficiently.

5.2 Feature Importance

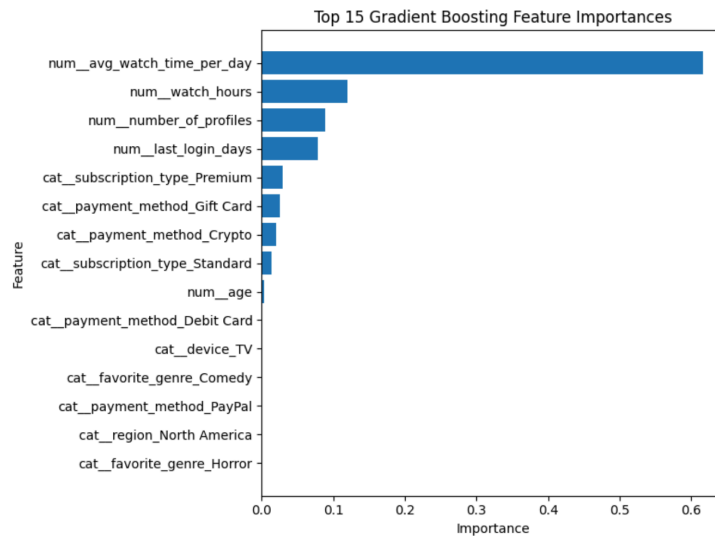


Figure 15: Gradient Boosting Feature Importances

After identifying the best-performing model, which is Gradient Boosting, we analyzed features that were driving the predictions. According to the bar chart, it shows that average watch time per day is the most important predictor with 0.6146, followed by watch hours 0.1204, number of profiles 0.0893, and last login days 0.0780. As such, it suggests that churn is largely associated with behavioral engagement, how the customer behaves on Netflix, rather than static customer characteristics. Customers who spend less time using the service, or have not logged in for a long time, are much more likely to leave the subscription. The number of profiles may also indicate that accounts used by more than one person are more engaged with the platform and less likely to churn compared to one profile usage.

5.3 Decision Tree Visualisation

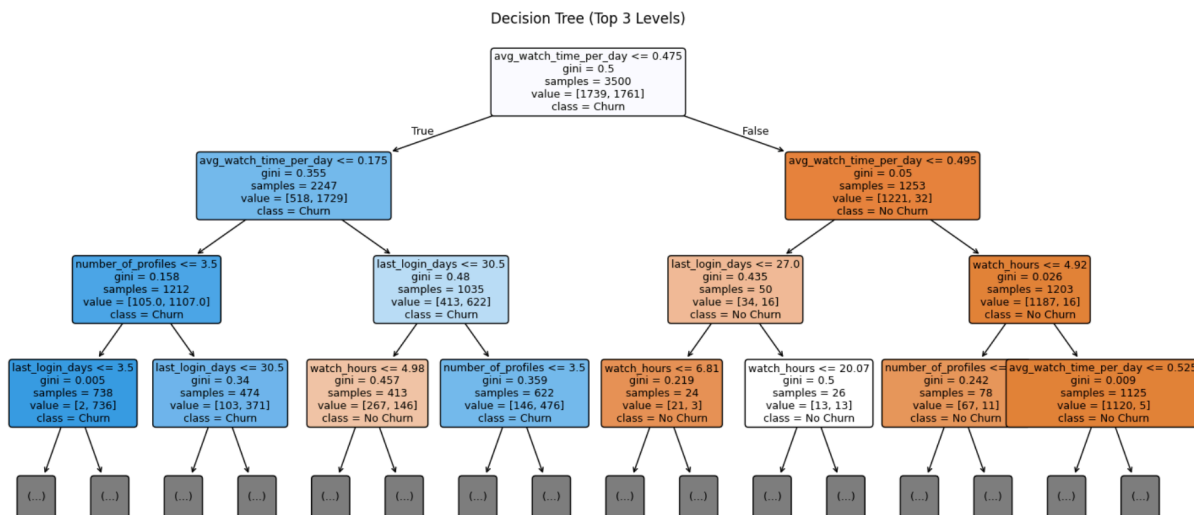


Figure 16: Decision Tree (Top 3 Levels)

Though we did not choose Decision Tree as the final selected model, its structure visualisation supports our interpretation. In the first split, it is based on average watch time per day, which shows that this feature is the most influential in distinguishing churned from retained customers. Followed by the last login days, watch hours, and the number of profiles. Accordingly, the Decision Tree shows that churn is associated with a combination of lower engagement and long inactivity, rather than depending on any one single feature.

5.4 Churn Rate by Features

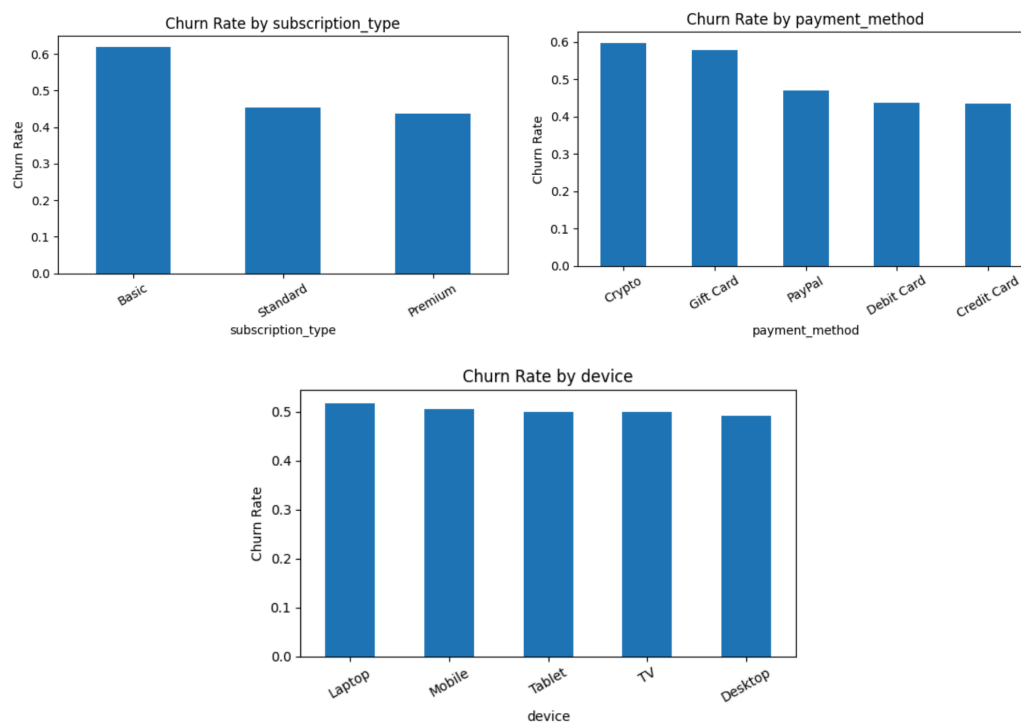


Figure 17: Churn Rate by key individual categories

To interpret how churn differs across categorical groups, we conducted churn rate charts across subscription type, payment method, and device. For subscription, Basic users have the highest churn rate at roughly 0.62, while Standard and Premium users are both lower. This suggests that customers on lower-tiered plans are more likely to leave, which may indicate lower perceived value among Basic users, although the dataset does not directly capture underlying reasons, such as film access, TV shows exclusive to higher-tiered plans, or advertisements during their watch. As a result, they are less committed to the service compared to other plans.

The payment method also shows that customers using Crypto and Gift Card payment have the highest churn rates at about 0.6 and 0.58, while PayPal, Debit Card, and Credit Card are lower subsequently. It demonstrates that more conventional payment methods reflect more long-term usage, which may suggest a relationship with shorter-term usage patterns.

By contrast, device type does not appear to have a significant relationship to the churn prediction since they are all clustered close together. This is supported by the feature importances chart of Gradient Boosting, where device features contribute very little to the final model. From the points given above, these findings show Netflix should focus more on engaging and in-account behavior when predicting churn rate.

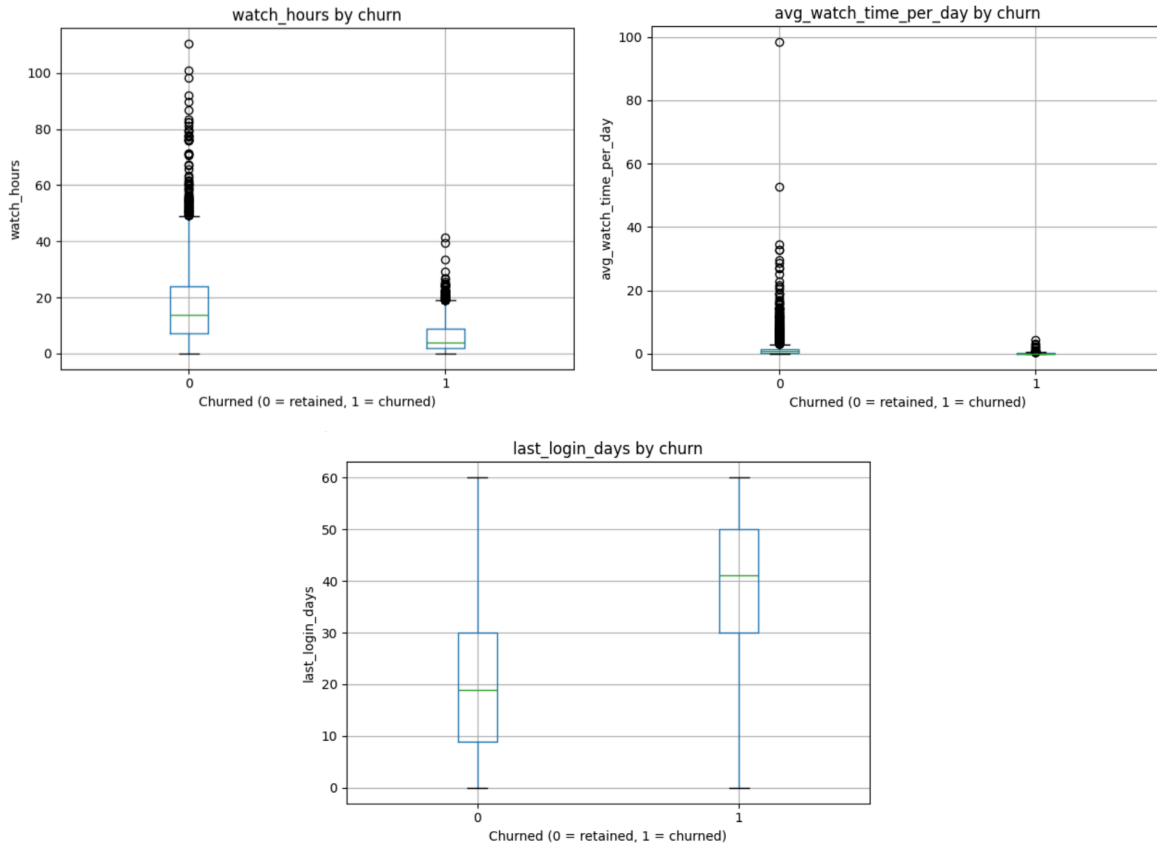


Figure 18: Distribution plots by quantitative variables

The distribution plots provide further insight into how churn differs across customers' in-app behaviors, where customers who stayed tend to have higher watch hours than customers who churned. The retained group also shows a wider spread, indicating that they vary in how much they log in and use the service. Additionally, average watch time per day of churned customers was close to zero, in contrast to higher values for retained customers. For the last few days of login, churned customers are at higher values, meaning they have not logged in to the service for a longer time. These findings support the idea that inactivity is one of the clearest warning signs to identify churn, especially on last login days, as it represents the strongest contrast.

6. Discussion

6.1 Key Findings

The results consistently point toward the conclusion: churn is essentially a problem of declining engagement, meaning customers gradually stop interacting with it, stop valuing it, and eventually cancel their membership. Gradient Boosting was selected as the best-performing model, achieving a test ROC-AUC of 0.9989 and a test accuracy of 0.9913, outperforming all other models, including Random Forest, Decision Tree, and the logistic regression variants. Looking at which features matter most in the model, we see that the strongest predictors of churn were average time watched per day, total watch hours, number of profiles, and last login days, all of which indicate how actively customers use the service. The average watch time per day variable was the most dominant with 61.6% of the model's total predictive power, which is almost five times more than the second feature, total watch hours.

6.2 Limitations

6.2.1 Data Limitations

As mentioned in Section 2, our dataset, retrieved from Kaggle, was synthetically generated. While it is structured, clean, and suitable to implement predictive modelling, it may not fully represent the complexity, variability, and realistic behavioral patterns of Netflix customers in real-world contexts. Second, the analysis captures the customer's behavior at a point in time, with a binary outcome of churn or no churn. The model could not capture how user activity changes over time. In practice, tracking how customers' behavior changes over subscription periods would likely yield clearer signals of churn. Finally, the dataset includes only basic information on the customers, without detailed demographic and usage context, such as completion of series or search history. This makes the model too generic to fully understand the true reasons behind customer churn.

6.2.2 Model Limitations

Based on our prediction, Gradient Boosting achieved an ROC-AUC of 0.999 and achieved clear classifications in the confusion matrix. This demonstrates that churned and retained customers in this dataset are highly separable and strong predictors within the sample. However, this near-perfect result can be attributed to the artificial nature of the dataset. Therefore, discrepancies between our dataset and actual Netflix data may be high, and our model performance on actual Netflix data would likely yield less predictive results. Therefore, the report may oversimplify the underlying drivers of churn to interpret the dataset's results and understand the pattern of Netflix customers' behavior.

6.3 Recommendations

We suggest that Netflix track key behavioral signals systemically, such as declining watch time, how often customers log in, and increasing user inactivity. Customers flagged at-risk of churn by the model could then be intervened with targeted strategies or propositions to make them stay. Given that the Gradient Boosting Model incorrectly predicted only 13 out of 1,500 observations, this suggests that these predictions are highly accurate within this dataset. This means Netflix can focus its efforts on at-risk customers without unnecessarily targeting those likely to stay anyway. Additionally, as the Basic subscribers showed the highest churn rate of 62%, Netflix could consider implementing a segmented customer retention strategy for each tiered plan across its subscriber base. For instance, Basic plan subscribers are more price-sensitive and more likely to reconsider the value of their remaining subscription, hence Netflix might want to focus on strengthening perceived values of their subscription through ways such as highlighting the breadth of available content and constantly reminding them of their relevant watching history. Netflix should also consider trials and first-month discounts for lower-tier plans to encourage initial subscription uptake among price-sensitive subscribers and improve its short-term retention rate.

7. Conclusion

This report was focused on predicting Netflix customer churn using seven different predictive models on a dataset of 5,000 customers. The gradient boosting model was the best-performing model among those tested, with an ROC-AUC of 0.9989 and very few misclassifications, indicating that it demonstrated the highest predictive performance among the models tested. The main finding of this report suggests that churn is primarily associated with behavioral disengagement. The strongest predictors were average daily watch time, total watch hours, number of profiles, and days since the last login, while factors like payment method and device type had negligible influence. If we look at the subscriber categories, users on the basic plan have a higher churn rate than those on the standard or premium plans. This can be explained by less accessible content, greater exposure to ads, and a sense that they are getting less value from the service. Netflix should consider taking proactive initiatives in monitoring early warning signs and customers' activities to identify at-risk users earlier and intervene in a timely and effective manner to reduce its cancellation rate. The firm should implement retention on each target customer segmentation to allocate retention resources more efficiently, strengthen customer value, and sustain subscription revenue in the long-term.

8. References

- Alavanza, M. (2025, April 24). *Singaporeans cancel Netflix subscriptions after price hike of up to S\$4 monthly* - Singapore news. The Independent Singapore News.
<https://theindependent.sg/singaporeans-cancel-netflix-subscriptions-after-price-hike-of-up-to-s4-monthly/>
- NIQ. (2024, June 1). *Netflix subscription tracker*. NIQ.
<https://nielseniq.com/global/en/insights/analysis/2024/netflix-subscription-tracker/>

9. Appendix

A1 Predictive Model Full Python Code

Netflix Customer Churn Prediction — Combined Notebook

This notebook combines the two earlier notebooks into one clean workflow:

1. Data loading and inspection
2. Exploratory data analysis
3. Feature preparation and preprocessing
4. Model building and tuning
5. Model evaluation and comparison
6. Interpretation and visualization
7. New customer prediction
8. Exporting results

```
# =====  
# 1. IMPORT LIBRARIES  
# =====  
import pandas as pd  
import numpy as np  
import matplotlib.pyplot as plt  
  
from sklearn.model_selection import (  
    train_test_split,  
    StratifiedKFold,  
    cross_validate,  
    cross_val_score,  
    GridSearchCV  
)  
from sklearn.compose import ColumnTransformer  
from sklearn.pipeline import Pipeline  
from sklearn.preprocessing import OneHotEncoder, StandardScaler  
  
from sklearn.linear_model import LogisticRegression  
from sklearn.tree import DecisionTreeClassifier, plot_tree, export_text  
from sklearn.ensemble import RandomForestClassifier, GradientBoostingClassifier  
  
from sklearn.metrics import (  
    accuracy_score,  
    precision_score,  
    recall_score,  
    f1_score,  
    roc_auc_score,  
    confusion_matrix,  
    classification_report,  
    roc_curve  
)
```

2. Load data and inspect the dataset

```
df = pd.read_csv("netflix_customer_churn.csv")

print("Shape:", df.shape)

print("\nFirst 5 rows:")
print(df.head())

print("\nColumn names:")
print(df.columns.tolist())

print("\nData types:")
print(df.dtypes)

print("\nMissing values by column:")
print(df.isnull().sum())

print("\nDuplicate rows:", df.duplicated().sum())

print("\nTarget distribution:")
print(df["churned"].value_counts())

print("\nTarget distribution (%):")
print(round(df["churned"].value_counts(normalize=True) * 100, 2))

print("\nCrosstab: subscription_type vs monthly_fee")
print(pd.crosstab(df["subscription_type"], df["monthly_fee"]))
```

3. Exploratory data analysis

```
numeric_cols_all = df.select_dtypes(include=["int64", "float64"]).columns.tolist()
numeric_cols_all.remove("churned")

categorical_cols_all = df.select_dtypes(include=["object"]).columns.tolist()

print("\nNumeric columns:")
print(numeric_cols_all)

print("\nCategorical columns:")
print(categorical_cols_all)

print("\nSummary statistics:")
print(df[numeric_cols_all].describe())

corr_df = df[numeric_cols_all + ["churned"]].corr()
print("\nCorrelation with churn:")
print(corr_df["churned"].sort_values(ascending=False))
```

```
# Boxplots for key numeric variables
key_numeric_vars = [
    "watch_hours",
    "avg_watch_time_per_day",
    "last_login_days",
    "number_of_profiles",
    "age"
]

for col in key_numeric_vars:
    plt.figure(figsize=(6, 4))
    df.boxplot(column=col, by="churned")
    plt.title(f"{col} by churn")
    plt.suptitle("")
    plt.xlabel("Churned (0 = retained, 1 = churned)")
    plt.ylabel(col)
    plt.tight_layout()
    plt.show()
```

```

# Churn rate by selected categorical variables
key_categorical_vars = [
    "subscription_type",
    "payment_method",
    "device",
    "region",
    "favorite_genre"
]

for col in key_categorical_vars:
    churn_rate = df.groupby(col)["churned"].mean().sort_values(ascending=False)
    plt.figure(figsize=(7, 4))
    churn_rate.plot(kind="bar")
    plt.title(f"Churn rate by {col}")
    plt.ylabel("Average churn rate")
    plt.xlabel(col)
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.show()

```

4. Prepare modelling dataset

```

# Drop non-predictive / redundant columns
# customer_id = identifier only
# monthly_fee = redundant with subscription_type
df_model = df.drop(columns=["customer_id", "monthly_fee"])

print("\nColumns used:")
print(df_model.columns.tolist())

# Features and target
X = df_model.drop(columns=["churned"])
y = df_model["churned"]

print("\nFeature matrix shape:", X.shape)
print("Target shape:", y.shape)

numeric_cols = X.select_dtypes(include=["int64", "float64"]).columns.tolist()
categorical_cols = X.select_dtypes(include=["object"]).columns.tolist()

print("\nNumeric predictors:")
print(numeric_cols)

print("\nCategorical predictors:")
print(categorical_cols)

```

```

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=42,
    stratify=y
)

print("\nTrain-test split completed.")
print("X_train shape:", X_train.shape)
print("X_test shape:", X_test.shape)
print("y_train shape:", y_train.shape)
print("y_test shape:", y_test.shape)

```

5. Preprocessing

```
# Linear models need scaling
linear_preprocessor = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_cols),
        ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), categorical_cols)
    ]
)

# Tree-based models do not need scaling
tree_preprocessor = ColumnTransformer(
    transformers=[
        ("num", "passthrough", numeric_cols),
        ("cat", OneHotEncoder(drop="first", handle_unknown="ignore"), categorical_cols)
    ]
)
```

6. Build model pipelines

```
log_reg_pipeline = Pipeline([
    ("preprocessor", linear_preprocessor),
    ("model", LogisticRegression(max_iter=1000, random_state=42))
])

ridge_pipeline = Pipeline([
    ("preprocessor", linear_preprocessor),
    ("model", LogisticRegression(
        penalty="l2",
        solver="lbfgs",
        max_iter=1000,
        random_state=42
    ))
])

lasso_pipeline = Pipeline([
    ("preprocessor", linear_preprocessor),
    ("model", LogisticRegression(
        penalty="l1",
        solver="liblinear",
        max_iter=1000,
        random_state=42
    ))
])

elastic_pipeline = Pipeline([
    ("preprocessor", linear_preprocessor),
    ("model", LogisticRegression(
        penalty="elasticnet",
        solver="saga",
        l1_ratio=0.5,
        max_iter=3000,
        random_state=42
    ))
])

fixed_dt_pipeline = Pipeline([
    ("preprocessor", tree_preprocessor),
    ("model", DecisionTreeClassifier(
        max_depth=5,
        min_samples_split=20,
        min_samples_leaf=10,
        random_state=42
    ))
])
```

```

dt_pipeline = Pipeline([
    ("preprocessor", tree_preprocessor),
    ("model", DecisionTreeClassifier(random_state=42))
])

rf_pipeline = Pipeline([
    ("preprocessor", tree_preprocessor),
    ("model", RandomForestClassifier(random_state=42))
])

gb_pipeline = Pipeline([
    ("preprocessor", tree_preprocessor),
    ("model", GradientBoostingClassifier(random_state=42))
])

```

7. Cross validation for baseline models

```

kfold = StratifiedKfold(n_splits=5, shuffle=True, random_state=42)

scoring = {
    "accuracy": "accuracy",
    "precision": "precision",
    "recall": "recall",
    "f1": "f1",
    "roc_auc": "roc_auc"
}

def evaluate_cv_model(pipeline, name, X_data, y_data):
    scores = cross_validate(
        pipeline,
        X_data,
        y_data,
        cv=kfold,
        scoring=scoring,
        return_train_score=False
    )

    print(f"\n=== {name} Cross Validation ===")
    for metric in scoring.keys():
        print(f"{metric}: {np.mean(scores['test_' + metric]):.4f}")

evaluate_cv_model(ridge_pipeline, "Logistic Regression (Ridge)", X, y)
evaluate_cv_model(lasso_pipeline, "Logistic Regression (Lasso)", X, y)
evaluate_cv_model(elastic_pipeline, "Logistic Regression (Elastic Net)", X, y)
evaluate_cv_model(fixed_dt_pipeline, "Decision Tree (Fixed Parameters)", X, y)

```

8. Hyperparameter tuning

```
lasso_param_grid = {
    "model__C": [0.01, 0.1, 1.0, 5.0, 10.0]
}

elastic_param_grid = {
    "model__C": [0.01, 0.1, 1.0, 5.0],
    "model__l1_ratio": [0.2, 0.5, 0.8]
}

dt_param_grid = {
    "model__max_depth": [3, 5, 7, 10, None],
    "model__min_samples_split": [2, 5, 10, 20],
    "model__min_samples_leaf": [1, 2, 5]
}

rf_param_grid = {
    "model__n_estimators": [100, 200],
    "model__max_depth": [5, 10, None],
    "model__min_samples_split": [2, 5, 10],
    "model__min_samples_leaf": [1, 2, 5]
}

gb_param_grid = {
    "model__n_estimators": [100, 200],
    "model__learning_rate": [0.01, 0.05, 0.1],
    "model__max_depth": [2, 3, 4]
}
```

```
# Baseline logistic regression CV AUC
log_reg_pipeline.fit(X_train, y_train)
log_cv_auc = cross_val_score(
    log_reg_pipeline,
    X_train,
    y_train,
    cv=5,
    scoring="roc_auc"
).mean()

# Ridge CV AUC
ridge_pipeline.fit(X_train, y_train)
ridge_cv_auc = cross_val_score(
    ridge_pipeline,
    X_train,
    y_train,
    cv=5,
    scoring="roc_auc"
).mean()

# Lasso tuning
lasso_grid = GridSearchCV(
    lasso_pipeline,
    lasso_param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
lasso_grid.fit(X_train, y_train)
best_lasso = lasso_grid.best_estimator_
print("Best Lasso params:", lasso_grid.best_params_)

# Elastic Net tuning
elastic_grid = GridSearchCV(
    elastic_pipeline,
    elastic_param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
elastic_grid.fit(X_train, y_train)
best_elastic = elastic_grid.best_estimator_
print("Best Elastic Net params:", elastic_grid.best_params_)
```

```

# Decision Tree tuning
dt_grid = GridSearchCV(
    dt_pipeline,
    dt_param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
dt_grid.fit(X_train, y_train)
best_dt = dt_grid.best_estimator_
print("Best Decision Tree params:", dt_grid.best_params_)

# Random Forest tuning
rf_grid = GridSearchCV(
    rf_pipeline,
    rf_param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
rf_grid.fit(X_train, y_train)
best_rf = rf_grid.best_estimator_
print("Best Random Forest params:", rf_grid.best_params_)

# Gradient Boosting tuning
gb_grid = GridSearchCV(
    gb_pipeline,
    gb_param_grid,
    cv=5,
    scoring="roc_auc",
    n_jobs=-1
)
gb_grid.fit(X_train, y_train)
best_gb = gb_grid.best_estimator_
print("Best Gradient Boosting params:", gb_grid.best_params_)

```

9. Evaluate all models on the test set

```

def evaluate_model(model, X_test, y_test, model_name, cv_auc=np.nan):
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[:, 1]

    return {
        "model": model_name,
        "cv_auc": cv_auc,
        "test_accuracy": accuracy_score(y_test, y_pred),
        "test_precision": precision_score(y_test, y_pred),
        "test_recall": recall_score(y_test, y_pred),
        "test_f1": f1_score(y_test, y_pred),
        "test_roc_auc": roc_auc_score(y_test, y_prob),
        "confusion_matrix": confusion_matrix(y_test, y_pred)
    }

results_list = []

results_list.append(
    evaluate_model(log_reg_pipeline, X_test, y_test, "Logistic Regression", log_cv_auc)
)
results_list.append(
    evaluate_model(ridge_pipeline, X_test, y_test, "Ridge Logistic Regression", ridge_cv_auc)
)
results_list.append(
    evaluate_model(best_lasso, X_test, y_test, "Lasso Logistic Regression", lasso_grid.best_score_)
)
results_list.append(
    evaluate_model(best_elastic, X_test, y_test, "Elastic Net Logistic Regression", elastic_grid.best_score_)
)
results_list.append(
    evaluate_model(best_dt, X_test, y_test, "Decision Tree", dt_grid.best_score_)
)
results_list.append(
    evaluate_model(best_rf, X_test, y_test, "Random Forest", rf_grid.best_score_)
)
results_list.append(
    evaluate_model(best_gb, X_test, y_test, "Gradient Boosting", gb_grid.best_score_)
)

results_df = pd.DataFrame(results_list).sort_values(by="test_roc_auc", ascending=False)

print("\nModel comparison table:")
print(results_df)

```

```

best_model_name = results_df.iloc[0]["model"]
print("\nBest model based on Test ROC-AUC:", best_model_name)

model_dict = {
    "Logistic Regression": log_reg_pipeline,
    "Ridge Logistic Regression": ridge_pipeline,
    "Lasso Logistic Regression": best_lasso,
    "Elastic Net Logistic Regression": best_elastic,
    "Decision Tree": best_dt,
    "Random Forest": best_rf,
    "Gradient Boosting": best_gb
}

best_model = model_dict[best_model_name]

y_pred_best = best_model.predict(X_test)
y_prob_best = best_model.predict_proba(X_test)[:, 1]

print("\nClassification report for best model:")
print(classification_report(y_test, y_pred_best))

print("Confusion matrix for best model:")
print(confusion_matrix(y_test, y_pred_best))

```

10. ROC curve comparison

```

plt.figure(figsize=(8, 6))

for model_name, model in model_dict.items():
    y_prob = model.predict_proba(X_test)[:, 1]
    fpr, tpr, _ = roc_curve(y_test, y_prob)
    auc_score = roc_auc_score(y_test, y_prob)
    plt.plot(fpr, tpr, label=f"{model_name} (AUC = {auc_score:.3f})")

plt.plot([0, 1], [0, 1], linestyle="--")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("ROC Curve Comparison")
plt.legend()
plt.tight_layout()
plt.show()

```

11. Model interpretation

```

# Logistic regression coefficients
log_preprocessor = log_reg_pipeline.named_steps["preprocessor"]
log_model = log_reg_pipeline.named_steps["model"]

feature_names_log = log_preprocessor.get_feature_names_out()
coefficients_log = log_model.coef_[0]

coef_df_log = pd.DataFrame({
    "feature": feature_names_log,
    "coefficient": coefficients_log
})

coef_df_log["abs_coefficient"] = coef_df_log["coefficient"].abs()
coef_df_log = coef_df_log.sort_values(by="abs_coefficient", ascending=False)

print("\nTop Logistic Regression coefficients:")
print(coef_df_log.head(15))

```

```

# Lasso coefficients
lasso_preprocessor = best_lasso.named_steps["preprocessor"]
lasso_model = best_lasso.named_steps["model"]

feature_names_lasso = lasso_preprocessor.get_feature_names_out()
coefficients_lasso = lasso_model.coef_[0]

coef_df_lasso = pd.DataFrame({
    "feature": feature_names_lasso,
    "coefficient": coefficients_lasso
})
coef_df_lasso["abs_coefficient"] = coef_df_lasso["coefficient"].abs()
coef_df_lasso = coef_df_lasso.sort_values(by="abs_coefficient", ascending=False)

print("\nTop Lasso coefficients:")
print(coef_df_lasso.head(15))

```

```

# Decision tree rules
dt_preprocessor = best_dt.named_steps["preprocessor"]
dt_model = best_dt.named_steps["model"]

feature_names_dt = dt_preprocessor.get_feature_names_out()

tree_rules = export_text(
    dt_model,
    feature_names=list(feature_names_dt)
)

print("\nDecision Tree rules:")
print(tree_rules)

```

```

# Decision tree plots
clean_feature_names_dt = [
    name.replace("num__", "").replace("cat__", "")
    for name in feature_names_dt
]

plt.figure(figsize=(26, 14))
plot_tree(
    dt_model,
    feature_names=clean_feature_names_dt,
    class_names=["Not Churned", "Churned"],
    filled=True,
    rounded=True,
    fontsize=8
)
plt.title("Decision Tree for Customer Churn Prediction")
plt.show()

plt.figure(figsize=(22, 10))
plot_tree(
    dt_model,
    feature_names=clean_feature_names_dt,
    class_names=["Not Churned", "Churned"],
    filled=True,
    rounded=True,
    fontsize=8,
    max_depth=3
)
plt.title("Decision Tree for Customer Churn Prediction (Top 3 Levels)")
plt.show()

```

```

# Random forest feature importance
rf_preprocessor = best_rf.named_steps["preprocessor"]
rf_model = best_rf.named_steps["model"]

feature_names_rf = rf_preprocessor.get_feature_names_out()
importances_rf = rf_model.feature_importances_

rf_importance_df = pd.DataFrame({
    "feature": feature_names_rf,
    "importance": importances_rf
}).sort_values(by="importance", ascending=False)

print("\nTop Random Forest feature importances:")
print(rf_importance_df.head(15))

top_rf = rf_importance_df.head(15)

plt.figure(figsize=(8, 6))
plt.barh(top_rf["feature"], top_rf["importance"])
plt.gca().invert_yaxis()
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Top 15 Random Forest Feature Importances")
plt.tight_layout()
plt.show()

```

```

# Gradient boosting feature importance
gb_preprocessor = best_gb.named_steps["preprocessor"]
gb_model = best_gb.named_steps["model"]

feature_names_gb = gb_preprocessor.get_feature_names_out()
importances_gb = gb_model.feature_importances_

gb_importance_df = pd.DataFrame({
    "feature": feature_names_gb,
    "importance": importances_gb
}).sort_values(by="importance", ascending=False)

print("\nTop Gradient Boosting feature importances:")
print(gb_importance_df.head(15))

top_gb = gb_importance_df.head(15)

plt.figure(figsize=(8, 6))
plt.barh(top_gb["feature"], top_gb["importance"])
plt.gca().invert_yaxis()
plt.xlabel("Importance")
plt.ylabel("Feature")
plt.title("Top 15 Gradient Boosting Feature Importances")
plt.tight_layout()
plt.show()

```

12. Check train versus test ROC-AUC

```

def train_test_auc(model, X_train, y_train, X_test, y_test, model_name):
    train_prob = model.predict_proba(X_train)[:, 1]
    test_prob = model.predict_proba(X_test)[:, 1]

    train_auc = roc_auc_score(y_train, train_prob)
    test_auc = roc_auc_score(y_test, test_prob)

    print(f"\n{model_name}")
    print(f"Train ROC-AUC: {train_auc:.4f}")
    print(f"Test ROC-AUC: {test_auc:.4f}")

for name, model in model_dict.items():
    train_test_auc(model, X_train, y_train, X_test, y_test, name)

```

13. Predict churn for a new customer

```
new_customer = pd.DataFrame([{
    "age": 35,
    "gender": "Female",
    "subscription_type": "Premium",
    "watch_hours": 40,
    "last_login_days": 12,
    "region": "Asia",
    "device": "Mobile",
    "payment_method": "Credit Card",
    "number_of_profiles": 3,
    "avg_watch_time_per_day": 1.8,
    "favorite_genre": "Drama"
}])

new_pred = best_model.predict(new_customer)[0]
new_prob = best_model.predict_proba(new_customer)[0, 1]

print("\n=== New Customer Prediction ===")
print("Predicted churn class:", new_pred)
print("Predicted churn probability:", round(new_prob, 4))

if new_pred == 1:
    print("Interpretation: This customer is predicted to churn.")
else:
    print("Interpretation: This customer is predicted to stay.")
```

14. Save outputs

```
results_df.to_csv("model_comparison_results.csv", index=False)
coef_df_log.to_csv("logistic_regression_coefficients.csv", index=False)
coef_df_lasso.to_csv("lasso_coefficients.csv", index=False)
rf_importance_df.to_csv("random_forest_feature_importance.csv", index=False)
gb_importance_df.to_csv("gradient_boosting_feature_importance.csv", index=False)

print("\nAll output files saved successfully.")
```